

AIE683 Machine Learning

Week 11: Time Series Forecasting

Dr. Hakan Emekci

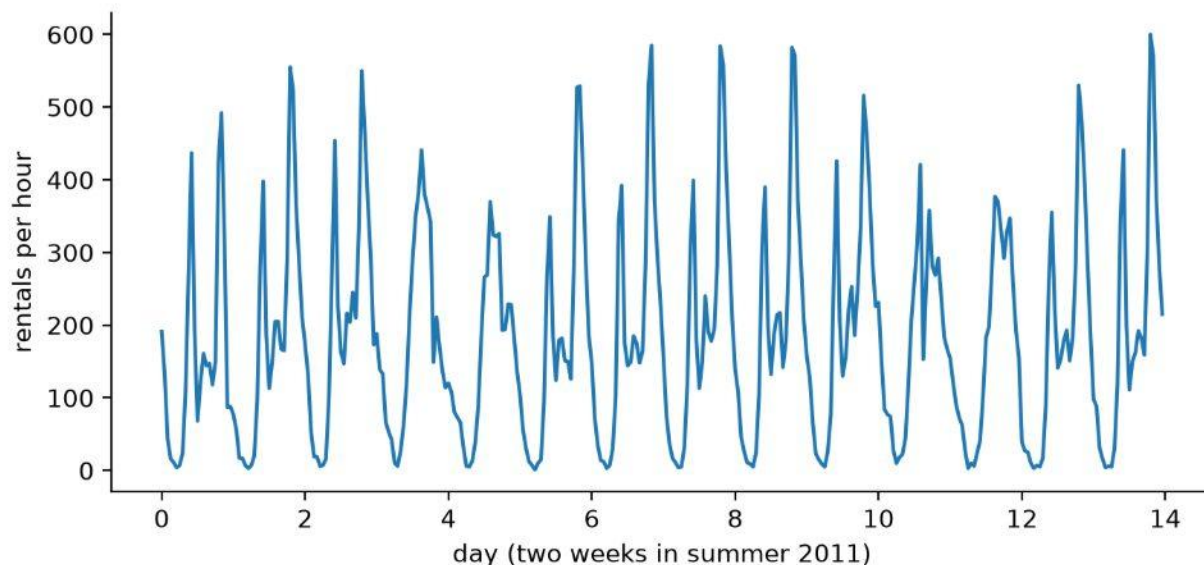
Hacettepe University, Graduate School of Informatics

Plan for today (3 hours)

1. **Tasks and data handling** (30 min): what forecasting is, pandas time indexes, resampling
2. **Classical forecasting** (40 min): stationarity, autocorrelation, decomposition, AR / ARIMA / ETS
3. **Forecasting as supervised learning** (60 min): lag and calendar features, `TimeSeriesSplit`, gradient boosting, backtesting, metrics
4. **Multi-step forecasts and uncertainty** (30 min): recursive vs. direct, quantile loss
5. **Tracking and packaging** (20 min): MLflow, the pipeline we serve in Week 12

Three **Try it** activities along the way; the notebook is `lab.ipynb`.

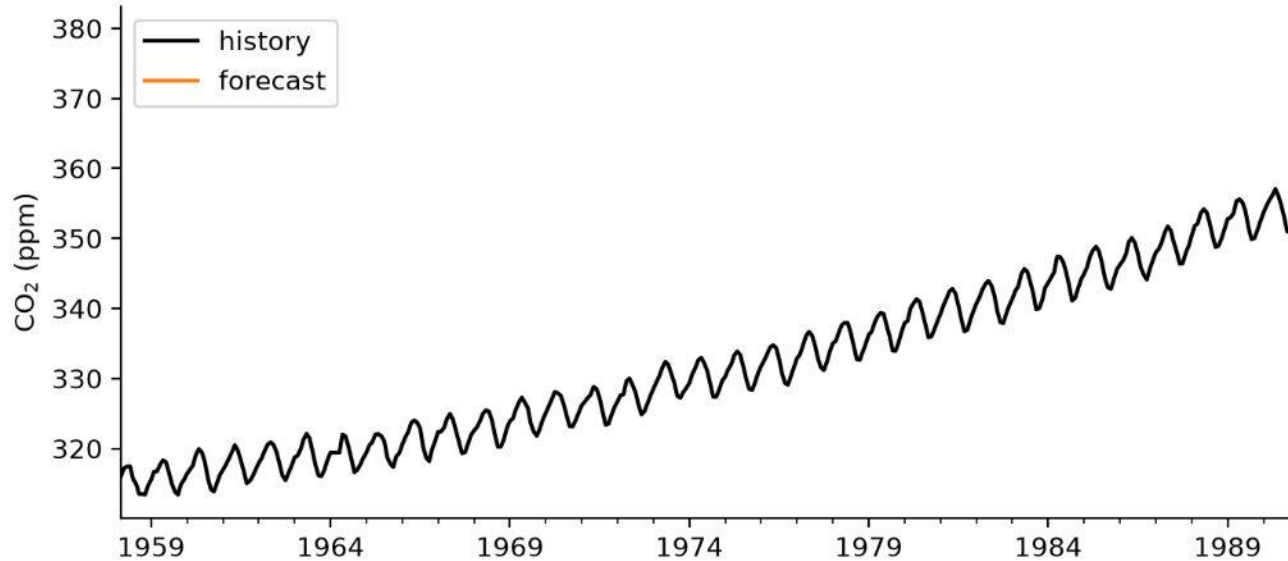
A forecasting problem worth solving



- Capital Bikeshare, Washington D.C.: rentals per hour, 2011–2012
- Every evening the operator decides where trucks move bikes and how many staff work tomorrow
- Too few bikes at 8 am: lost rides and angry commuters. Too many: wasted truck hours
- We need **tomorrow's demand, hour by hour**, before tomorrow starts

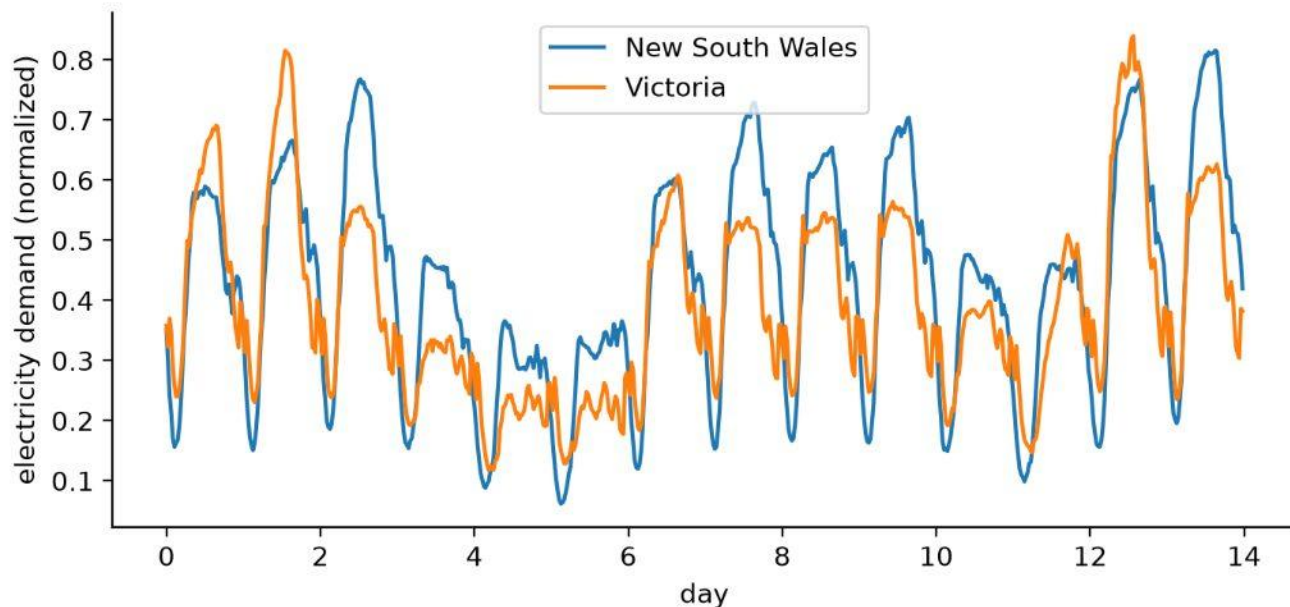
Tasks

1d forecasting



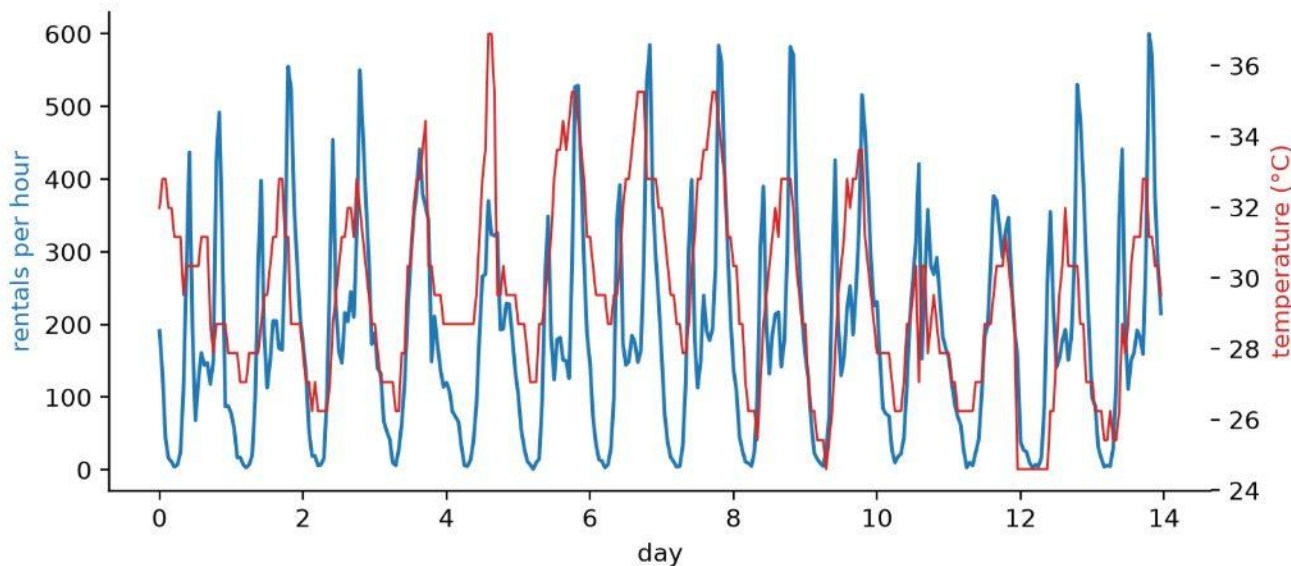
- Only the history of one series, e.g. monthly CO₂ at Mauna Loa

Nd forecasting



- Several related series forecast together, e.g. electricity demand in two neighbouring states

Feature-based forecasting (explanatory variables)



- Target: rentals per hour. Known inputs: calendar, weather, holidays

What's different?

- Forecasting: the rows are **not i.i.d.**; neighbouring observations are correlated, and the process drifts over time
- We always learn from the past and predict the future, so **evaluation must respect time**
- Classification: series of different lengths, feature extraction is the main work

Preliminaries

Measurement frequencies

$((x_{t_1}, y_{t_1}), \dots, (x_{t_m}, y_{t_m})), \quad t_1 < t_2 < \dots < t_m$

Equally spaced:

$$t_{i+1} - t_i = \text{const}$$

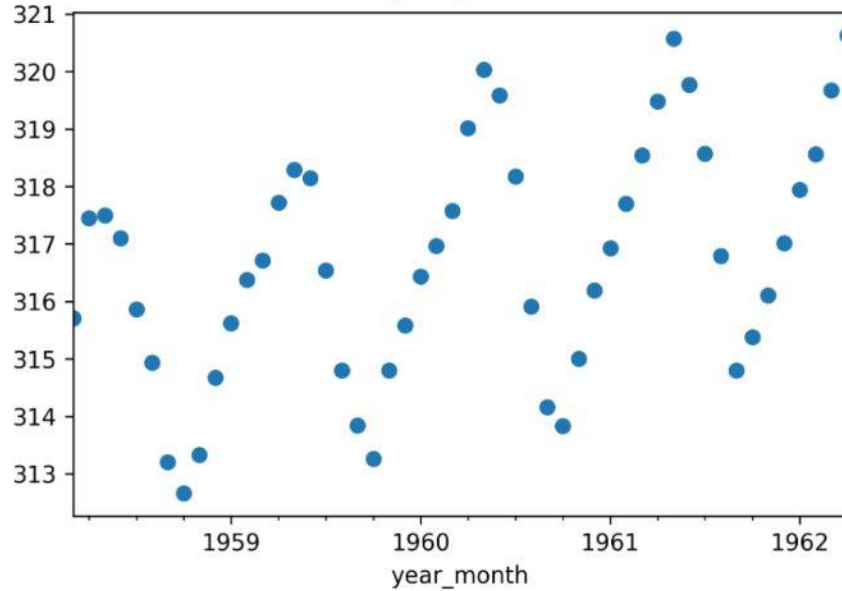
Unevenly spaced:

$$t_{i+1} - t_i \quad \text{varies}$$

aka "point processes" in statistics

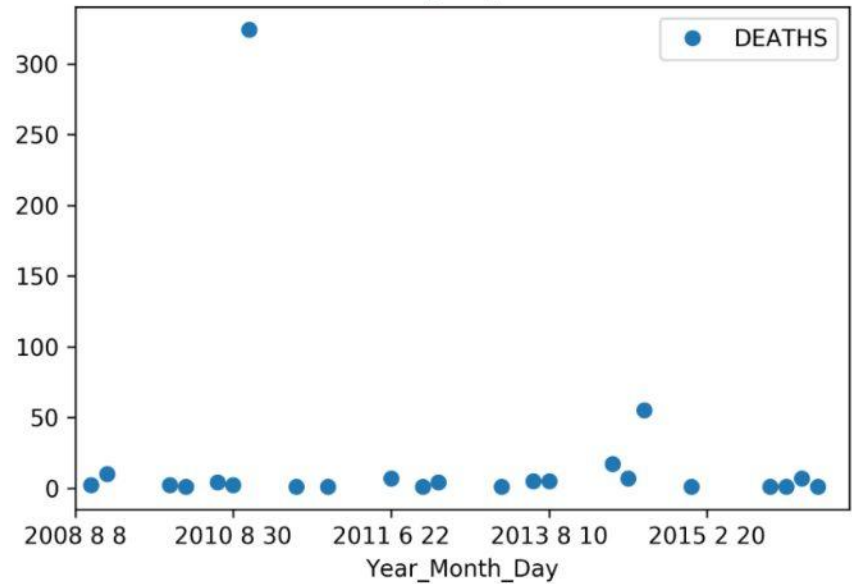
Examples

Evenly spaced



CO₂ measurements

Unevenly spaced



Volcanic eruption deaths

Time series in pandas

Parse dates, set a DatetimeIndex

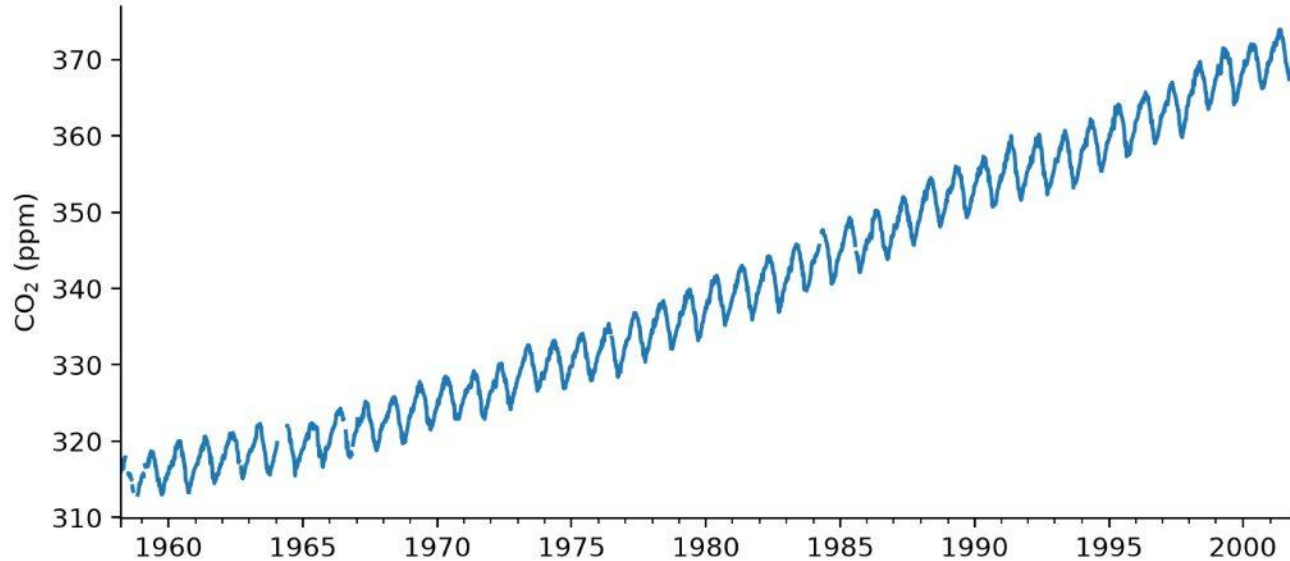
```
from sklearn.datasets import fetch_openml
co2 = fetch_openml(data_id=41187, as_frame=True).frame # Mauna Loa, weekly
co2[["year", "month", "day", "co2"]].head(3)
```

	year	month	day	co2
0	1958	3	29	316.1
1	1958	4	5	317.3
2	1958	4	12	317.6

```
co2["date"] = pd.to_datetime(co2[["year", "month", "day"]])
ppm = co2.set_index("date")["co2"]
ppm.index.to_series().diff().value_counts().head(3)
```

7 days	2202
14 days	14
28 days	2

Mauna Loa CO₂



```
weekly = ppm.asfreq("W-SAT") # complete weekly grid, missing weeks become NaN
weekly.isna().sum()          # 59
```

Resampling

```
monthly = weekly.resample("MS").mean() # MS = month start  
monthly.head(4)
```

```
date  
1958-03-01    316.100000  
1958-04-01    317.200000  
1958-05-01    317.433333  
1958-06-01         NaN  
Freq: MS, Name: co2, dtype: float64
```

- `resample` groups by time period; aggregate with `mean`, `sum`, `max`, ...
- Common aliases: `"h"`, `"D"`, `"W"`, `"MS"`, `"QS"`, `"YS"`; multiples like `"3YS"`

Classical forecasting

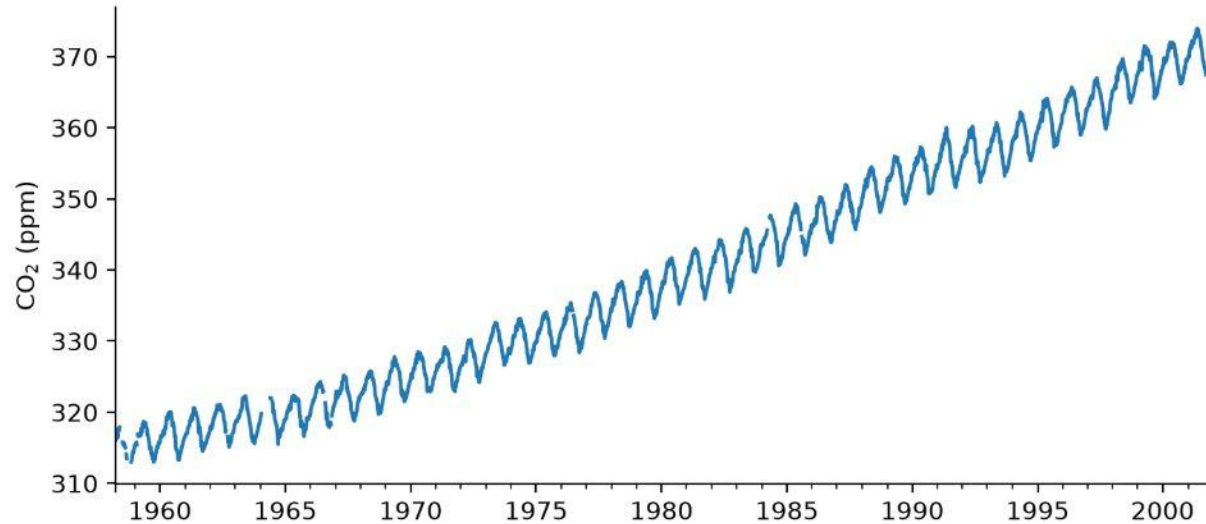
Stationarity

A series is (weakly) stationary if for all t and lags k

$$\begin{aligned} \mathbb{E}[x_t] &= \mu, \quad \text{and} \quad \text{Var}[x_t] = \sigma^2, \\ \text{Cov}[x_t, x_{t+k}] &= \gamma_k \end{aligned}$$

- Mean, variance and lag- k covariance do not depend on time
- Required by ARMA-type models; trends and seasonality violate it

CO₂ is not stationary



- The mean increases: a trend
- A yearly cycle: seasonality

Autocorrelation

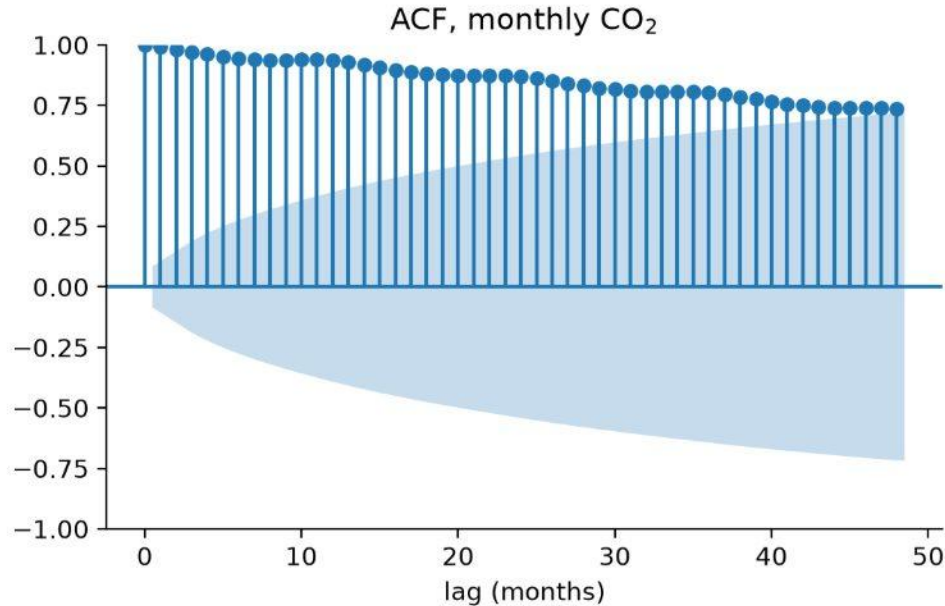
$$\rho_k = \frac{E[(x_t - \mu)(x_{t+k} - \mu)]}{\sigma^2}$$

```
monthly = monthly.ffill()  
monthly.autocorr(lag=1), monthly.autocorr(lag=6), monthly.autocorr(lag=12)
```

(0.998, 0.973, 0.999)

Autocorrelation function (ACF)

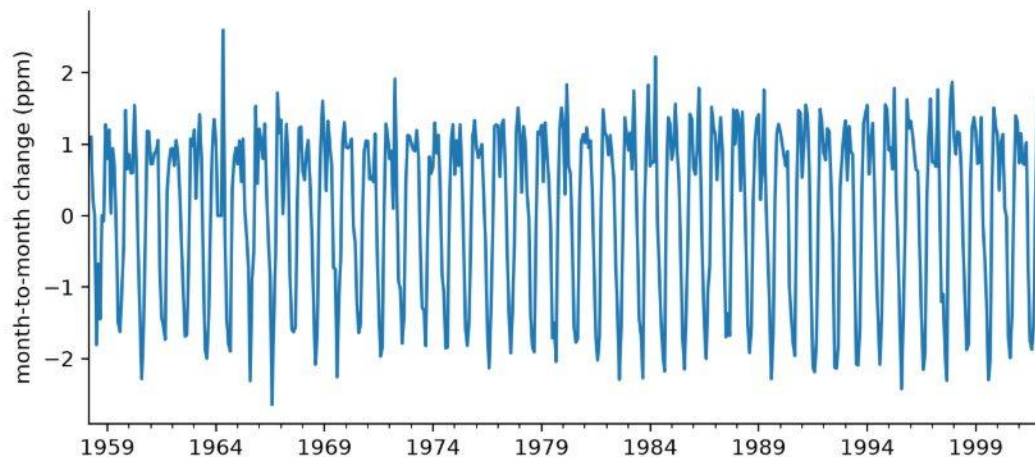
```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(monthly, lags=48)
```



De-trending by differencing

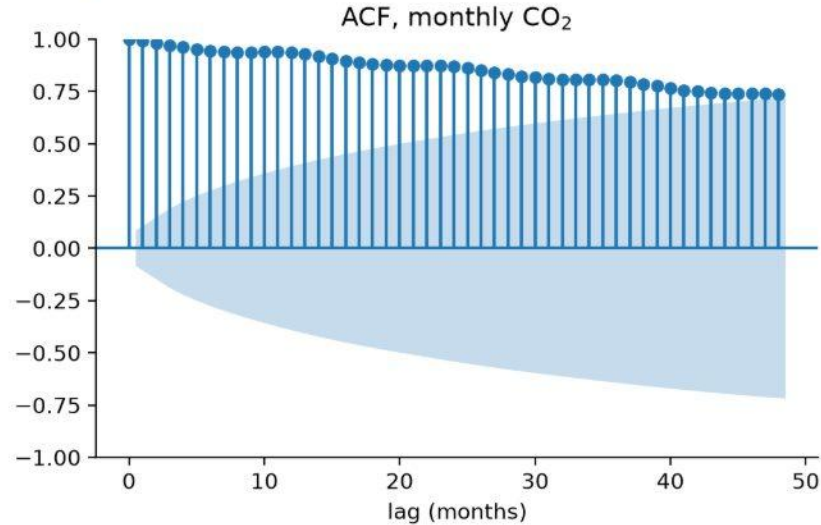
$$x'_t = x_t - x_{t-1}$$

```
monthly.diff().plot()
```

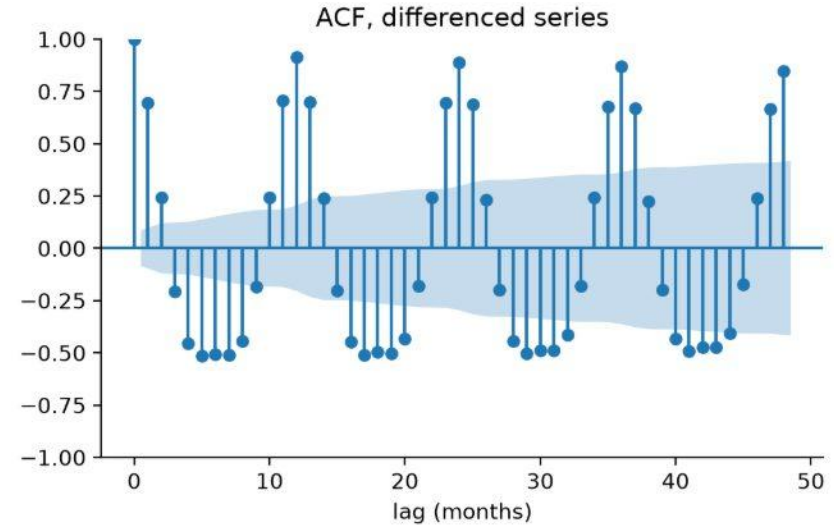


ACF: original vs. differenced

Original series

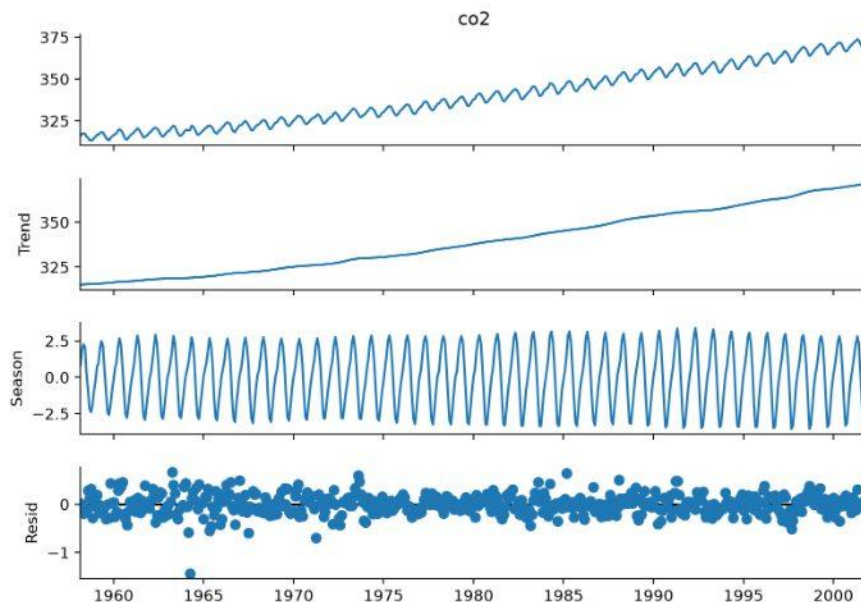


Differenced series



Decomposition: trend + season + remainder

```
from statsmodels.tsa.seasonal import STL
STL(monthly, period=12).fit().plot()
```



Autoregressive (AR) model

Order-p AR model with intercept:

$$x_t = c + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \dots + \phi_p x_{t-p} + \varepsilon_t$$

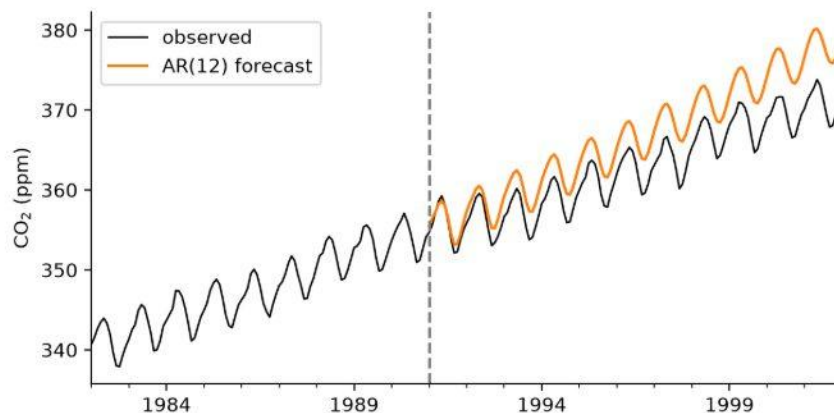
- Learn $c, \phi_1, \dots, \phi_p$ by least squares
- Forecast recursively: predictions become inputs for later steps

AR model with statsmodels

```
from statsmodels.tsa.ar_model import AutoARIMA
train = monthly[:"1990"]
test = monthly["1991":]
ar = AutoARIMA(train, lags=12, trend="c")
ar.params.round(3)
```

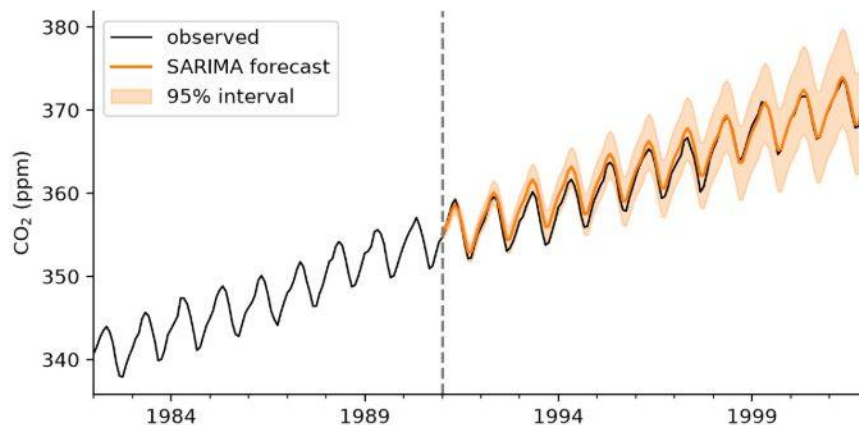
const	-3.524
co2.L1	0.836
co2.L2	-0.168
...	
co2.L11	0.382
co2.L12	0.072

```
pred = ar.predict(start=test.index[0],
                  end=test.index[-1])
```



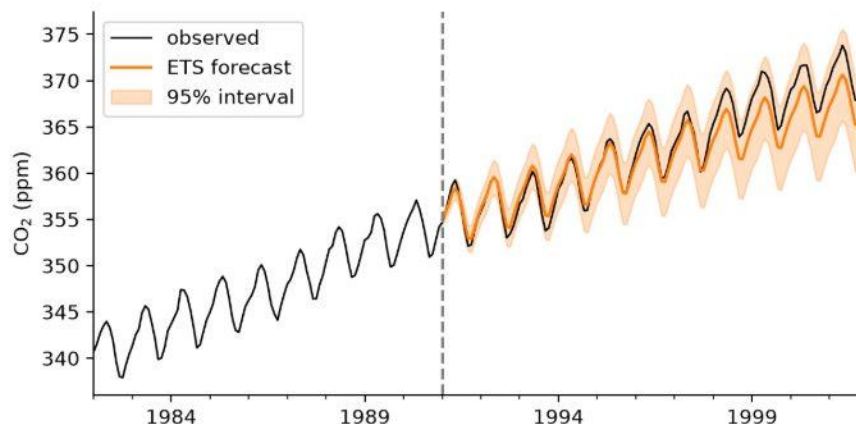
(S)ARIMA

```
from statsmodels.tsa.arima.model import ARIMA
sarima = ARIMA(train, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit()
fc = sarima.get_forecast(steps=len(test))
fc.predicted_mean, fc.conf_int(alpha=0.05)
```



Exponential smoothing (ETS)

```
from statsmodels.tsa.exponential_smoothing.ets import ETSModel  
ets = ETSModel(train, error="add", trend="add", seasonal="add", seasonal_periods=12)  
ets.get_prediction(start=test.index[0], end=test.index[-1]).summary_frame(alpha=0.0)
```



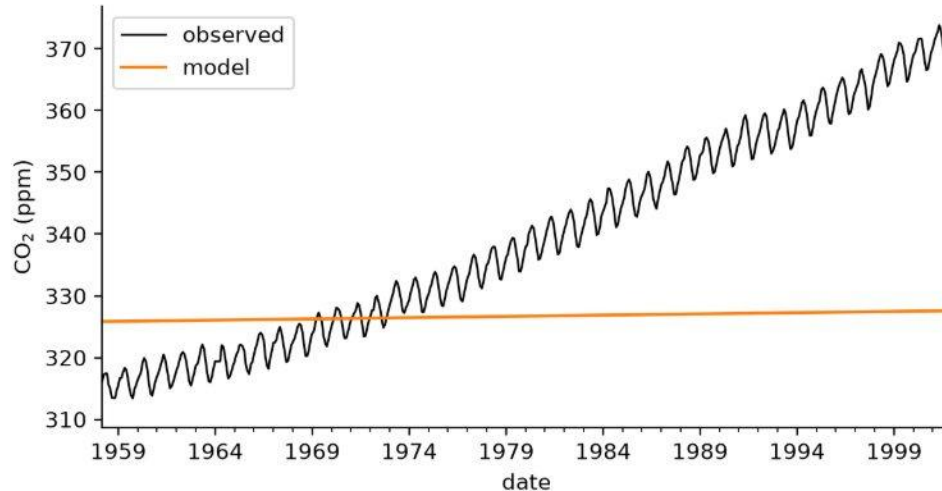
Always start with naive baselines

- **Naive:** $\hat{x}_{t+h} = x_t$ (the last value)
- **Seasonal naive:** $\hat{x}_{t+h} = x_{t+h-m}$ (same period last season, m = season length)
- Cost nothing, need no tuning, and are surprisingly hard to beat on noisy data
- A model that does not beat seasonal naive in a proper backtest is not ready for use

Forecasting as supervised learning

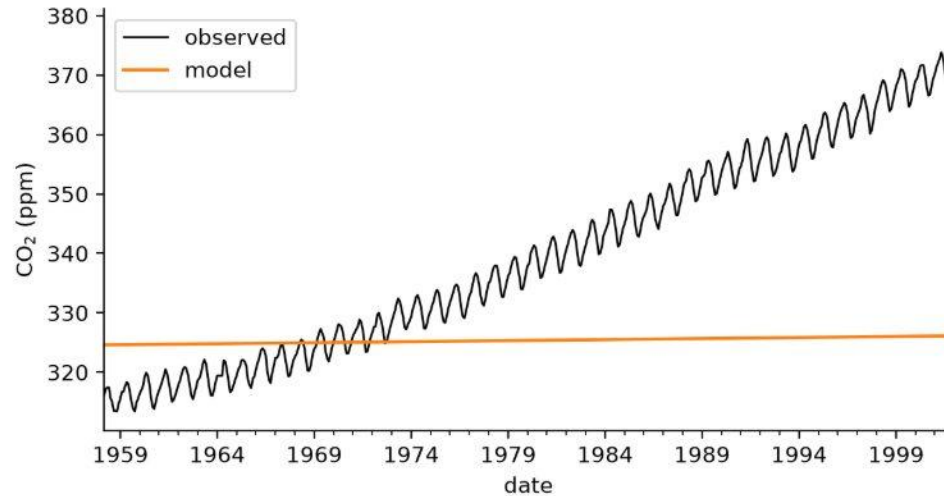
Regression on time: the trend

```
X = pd.DataFrame({"t": np.arange(len(monthly)) / 12,      # years since start  
                  "month": monthly.index.month}, index=monthly.index)  
lr = LinearRegression().fit(X_train[["t"]], train)
```



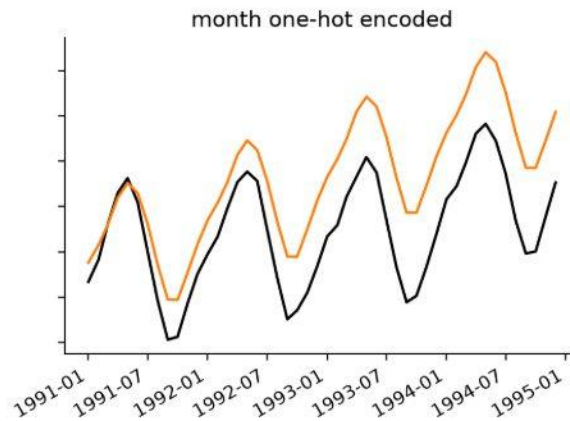
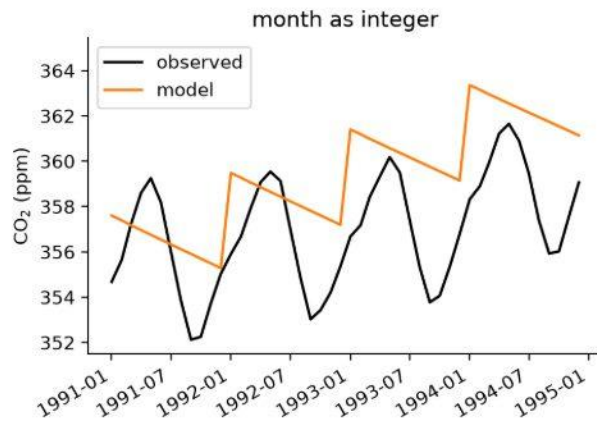
A quadratic trend

```
quad = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), LinearRegres  
quad.fit(X_train[["t"]], train)
```



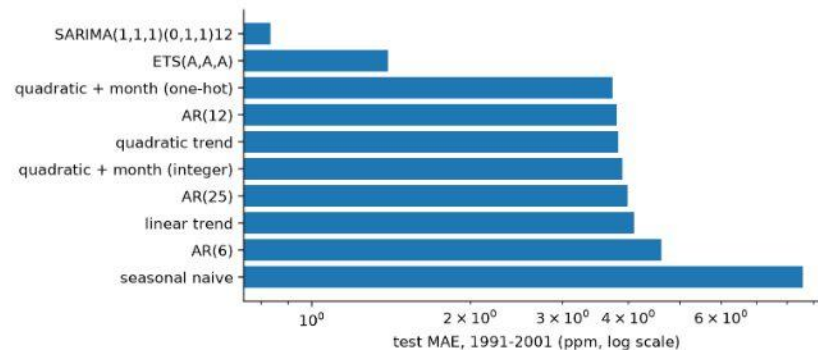
Adding the season: integer vs. one-hot month

```
month_ohe = ColumnTransformer([
    ("trend", PolynomialFeatures(2, include_bias=False), ["t"]),
    ("month", OneHotEncoder(sparse_output=False), ["month"]),
])
quad_month = make_pipeline(month_ohe, LinearRegression()).fit(X_train, train)
```

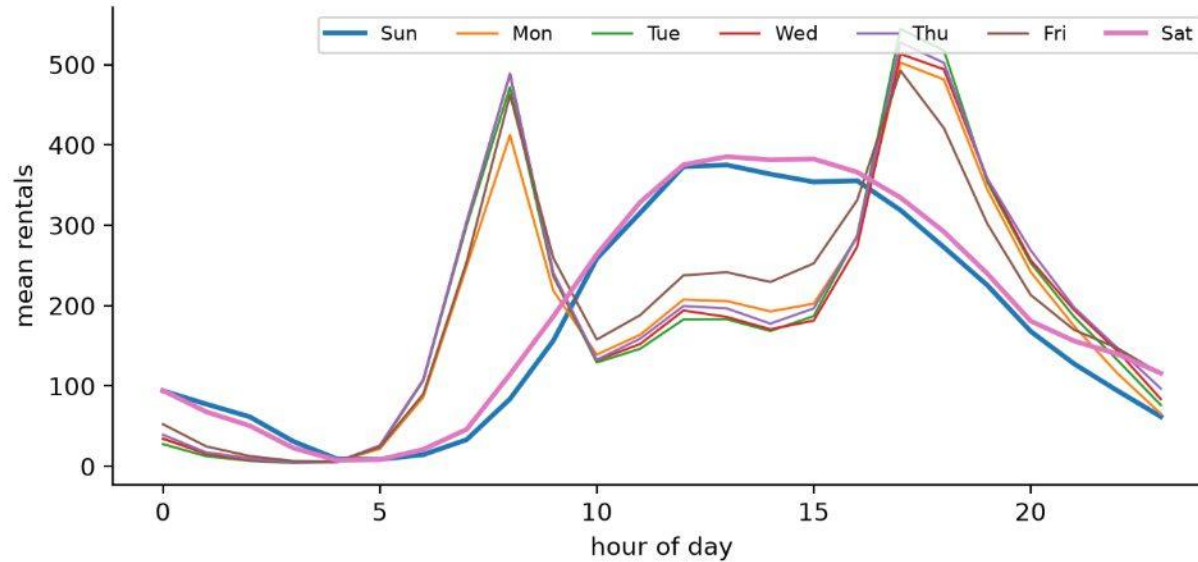


CO₂: test MAE over 1991–2001

model	MAE (ppm)
SARIMA(1,1,1)(0,1,1) ₁₂	0.83
ETS (A,A,A)	1.40
quadratic + month one-hot	3.72
AR(12)	3.80
quadratic trend	3.82
linear trend	4.09
seasonal naive	8.58



Back to bike sharing



- 17,379 hours; columns: season, year, month, hour, weekday, holiday, working day, weather, temperature, humidity, wind
- Two daily peaks on working days, one broad afternoon hump on weekends

Framing: one row per target hour

For a forecast made h hours ahead (here $h = 24$):

features: known at forecast time	target
calendar of the target hour (hour, weekday, holiday, month)	y_t
weather at the target hour (in production: a weather forecast)	
lags $y_{t-24}, y_{t-48}, y_{t-168}$ (all $\geq h$)	
rolling windows of the shifted series, e.g. mean of $y_{t-47}, \dots, y_{t-24}$	

```
shifted = y.shift(24)
XL = X.assign(lag_24=shifted, lag_48=y.shift(48), lag_168=y.shift(168),
              roll_mean_24=shifted.rolling(24).mean())
```

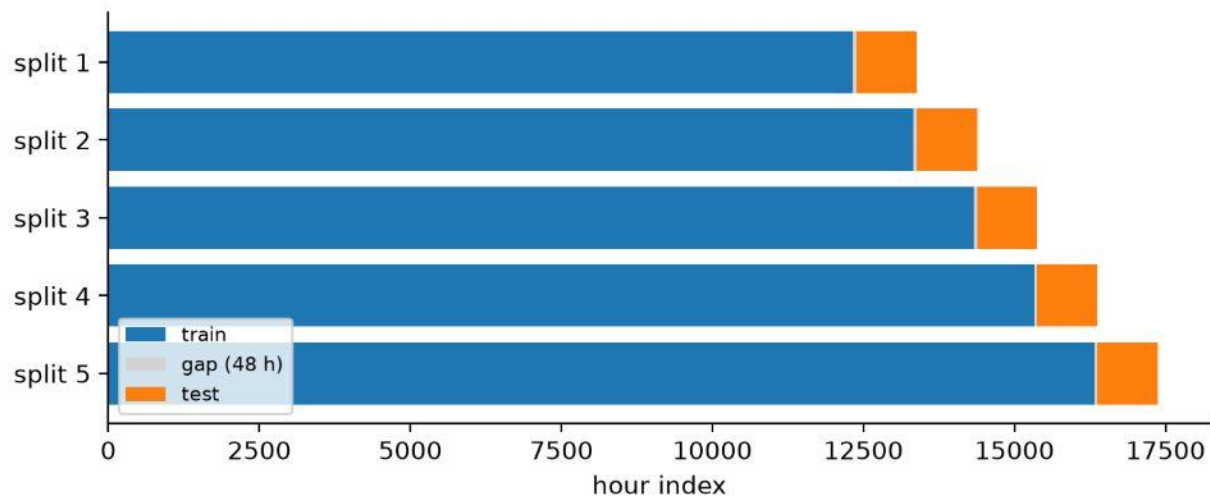
Why shuffled cross-validation lies

Recall Weeks 2–3: the split must mimic how the model will be used.

```
hgb = HistGradientBoostingRegressor(categorical_features="from_dtype")
cross_val_score(hgb, XL, y, cv=KFold(5, shuffle=True), scoring="neg_mean_absolute_e
cross_val_score(hgb, XL, y, cv=TimeSeriesSplit(5, test_size=1000, gap=48), ...)
```

CV scheme	MAE (rentals / hour)
shuffled KFold	29.7
TimeSeriesSplit	46.0

TimeSeriesSplit with a gap



- Train on the past, test on the next block; expanding window (or `max_train_size`)
- `gap` drops rows between train and test: needed when features or targets span windows

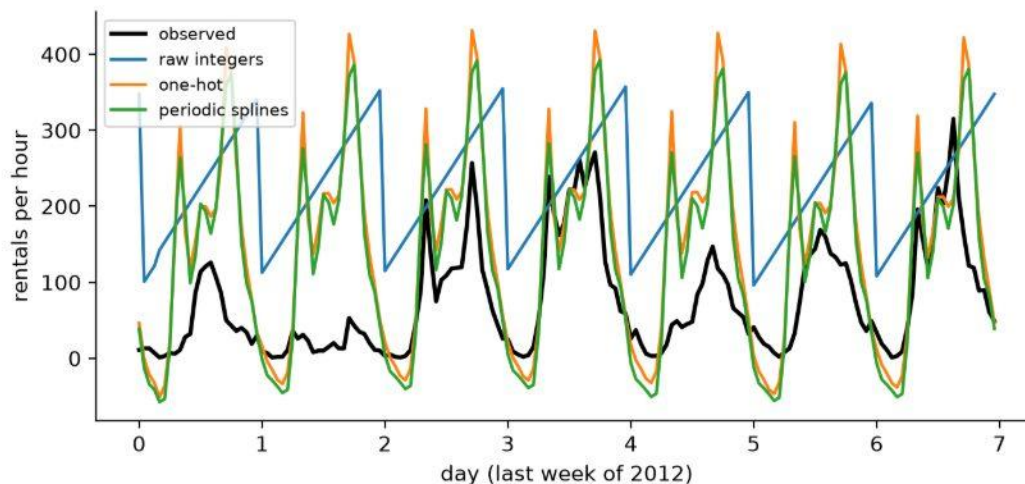
Try it 1: find the leak (10 min)

A colleague's feature table for the **day-ahead** forecast:

```
colleague = X.assign(lag_1=y.shift(1),  
                    lag_24=y.shift(24),  
                    smooth_24=y.rolling(24, center=True).mean())
```

1. Without running it: which features could not exist at 18:00 the day before?
2. Run it with `TimeSeriesSplit`. Why does it still look great?
3. Fix the table and compare (lab, "Try it 1")

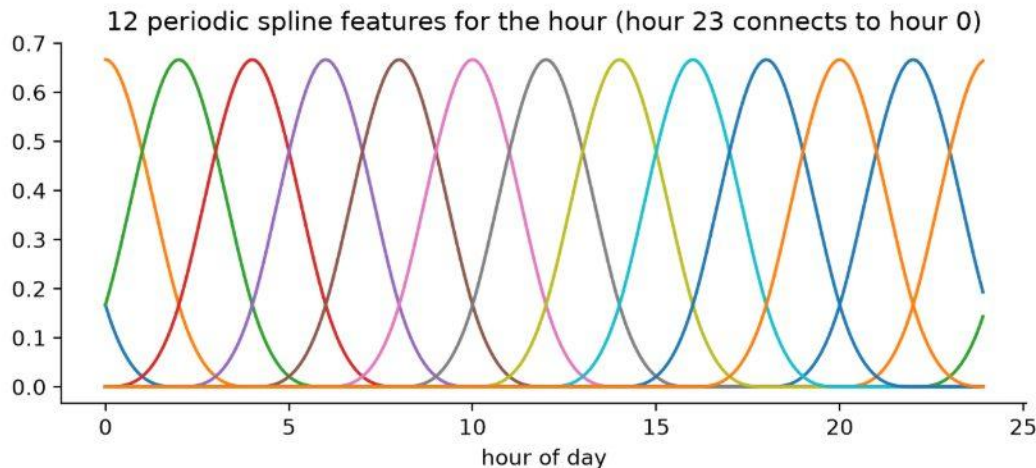
Calendar features for cyclic variables



Ridge with hour, weekday, month as	MAE
integers	158
one-hot	115
periodic splines	112

Periodic spline features

```
def periodic_spline(period, n_splines):  
    knots = np.linspace(0, period, n_splines + 1).reshape(-1, 1)  
    return SplineTransformer(degree=3, knots=knots, extrapolation="periodic")  
  
ColumnTransformer([("hour", periodic_spline(24, 12), ["hour"]),  
                  ("weekday", periodic_spline(7, 3), ["weekday"]),  
                  ("month", periodic_spline(12, 6), ["month"])], remainder="passth
```



Gradient boosting for forecasting

- `HistGradientBoostingRegressor`: native categoricals (`categorical_features="from_dtype"`), handles missing values (early lags)
- LightGBM / XGBoost: same idea, more options; the default in many forecasting competitions
- Learns interactions (hour $\times$ working day $\times$ weather) without feature engineering
- **Caveat: trees cannot extrapolate.** A tree never predicts above the largest training target. Detrend or difference the target, or predict a ratio, when there is a trend.
- Consider `loss="poisson"` for counts

The backtest: our referee

- `TimeSeriesSplit(n_splits=3, test_size=24 * 28)`: three 4-week test windows at the end of 2012
- ML models: refit per fold, predict the whole window (all features are ≥ 24 h old)
- Classical baselines: one 24-hour forecast per test day using only earlier data
 - naive, seasonal naive (24 h and 168 h), Holt-Winters ETS with a daily season (refit daily on 4 weeks)
- Same windows and metrics for every model, all logged to MLflow

Evaluation metrics

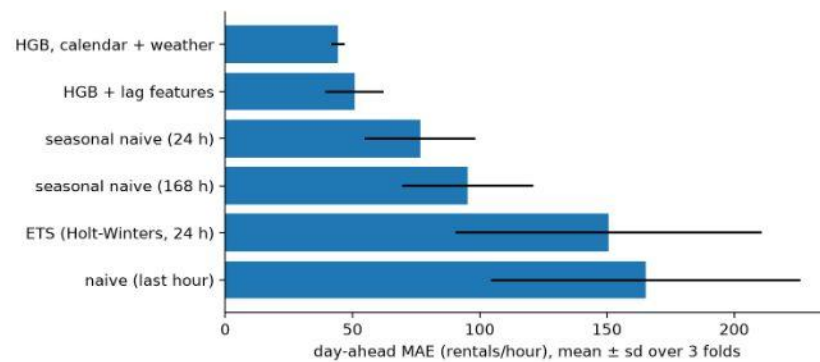
$$\text{MAE} = \frac{1}{n} \sum |y_t - \hat{y}_t| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum (y_t - \hat{y}_t)^2}$$

$$\begin{aligned} \text{MAPE} &= \frac{1}{n} \sum \left| \frac{y_t - \hat{y}_t}{y_t} \right| \\ \text{MASE} &= \frac{\text{MAE}}{\frac{1}{T-m} \sum_{t=m+1}^T |y_t - y_{t-m}|} \end{aligned}$$

- MAE: in units of the target, robust. RMSE: punishes large misses
- MAPE: undefined at $y = 0$, explodes for small y , asymmetric. Avoid for counts
- MASE: MAE relative to in-sample seasonal naive; < 1 beats the baseline, comparable across series

Backtest results (day-ahead, mean of 3 folds)

model	MAE	RMSE	MAPE	MASE
HGB, calendar + weather	44.3	68.1	0.72	0.68
LightGBM + lags	48.5	77.0	1.03	0.75
HGB + lags	51.5	80.6	1.21	0.80
seasonal naive 24 h	76.6	124.4	0.91	1.19
seasonal naive 168 h	95.3	156.6	2.02	1.48
ETS (Holt-Winters)	150.7	193.0	5.14	2.34
naive	165.3	209.0	6.16	2.57



Try it 2: predict the winner, then beat it (15 min)

1. Before running the backtest cell: write down which model you expect to win and by how much
2. Run it. Were you right? Why do lag features not help here?
3. Beat the best backtest MAE without changing the protocol. Ideas:
`loss="poisson"`, other lags, `learning_rate` / `max_iter`, a log target
(`TransformedTargetRegressor`)
4. Log your run with `log_backtest(...)`; we compare the leaderboard in MLflow at the end

What goes wrong in practice

- **Features you will not have:** observed weather instead of the forecast, lags shorter than the horizon, centered windows
- **Gaps in the time index:** `shift(24)` on rows is not "24 hours ago" when hours are missing. Reindex to a complete grid first
- **Scaling or imputing on the whole series** before splitting (Week 3 leakage, now along time)
- **Tuning on the test period:** hyperparameters chosen on the backtest you report. Hold out a final period
- **Regime changes:** holidays, new stations, a pandemic; no backtest on 2012 predicts 2020
- **Trend + trees:** flat forecasts above the training range

Multi-step forecasts and uncertainty

Multi-step strategies

Recursive (iterated): one 1-step model f ; feed predictions back

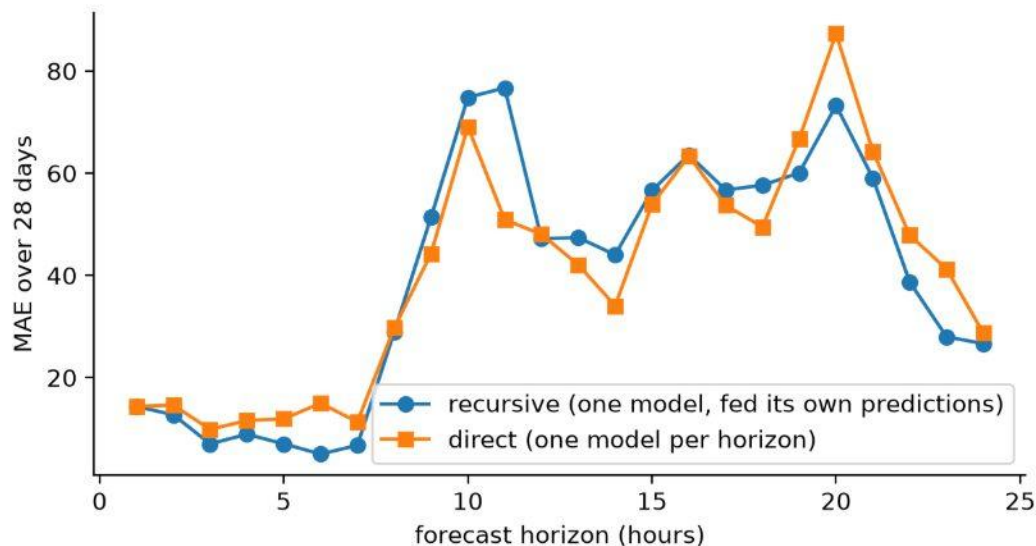
$$\hat{y}_{t+1} = f(y_t, y_{t-1}, \dots), \quad \hat{y}_{t+2} = f(\hat{y}_{t+1}, y_t, \dots), \quad \dots$$

Direct: one model per horizon h , trained only on lags $\geq h$

$$\hat{y}_{t+h} = f_h(y_t, y_{t-1}, \dots), \quad h = 1, \dots, H$$

- Recursive: one model, uses the freshest data, but errors compound
- Direct: no error feedback, but H models and each ignores the most recent values it cannot see
- Hybrids: one model with the horizon as a feature; multi-output models

Recursive vs. direct on bike sharing



- 28 daily origins in December 2012, gradient boosting with lags 1, 2, 3, 24, 168 (recursive) or $h, h + 1, h + 2, 24, 168$ (direct)
- Mean MAE over 24 horizons: about 40 for both

Prediction intervals with quantile loss

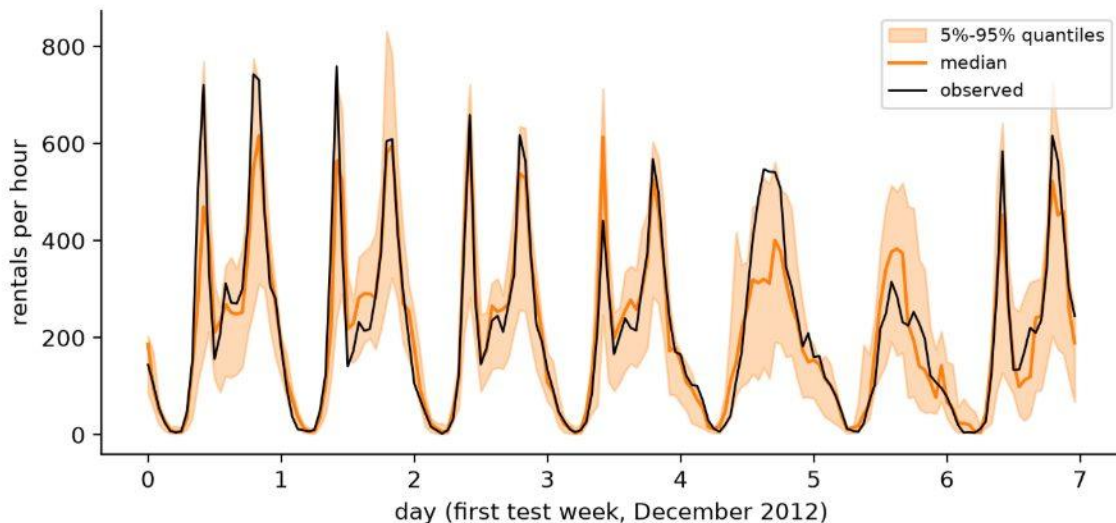
Pinball loss for quantile q :

$$L_q(y, \hat{y}) = \max \left(q (y - \hat{y}), (q - 1) (y - \hat{y}) \right)$$

```
lower = HistGradientBoostingRegressor(loss="quantile", quantile=0.05).fit(X_tr, y_tr)
upper = HistGradientBoostingRegressor(loss="quantile", quantile=0.95).fit(X_tr, y_tr)
coverage = np.mean((y_te >= lower.predict(X_te)) & (y_te <= upper.predict(X_te)))
```

- Minimizing L_q gives the conditional q -quantile; two models give a 90% interval
- Check **empirical coverage** on the backtest; evaluate with `mean_pinball_loss`

Quantile forecasts on the first test week



- With lag features: empirical coverage 0.89 for a nominal 90%, mean width 172 bikes
- Calendar + weather only (the lab): coverage 0.78, too narrow

Try it 3: is the interval right at every hour? (10 min)

- Overall coverage can be fine while rush hours are under-covered and nights over-covered
- In the lab: compute **coverage per hour of day** for the 90% interval and plot it
- Discuss: which hours fail? What would you change? (log target, lag features, conformal calibration per hour)

Tracking and packaging

Logging the backtest to MLflow

```
mlflow.set_experiment("week11-time-series")

def log_backtest(name, fold_metrics, model=None, features=None):
    with mlflow.start_run(run_name=name):
        mlflow.set_tags({"task": "day-ahead forecast", "cv": "TimeSeriesSplit(3 x 6"
        if model is not None:
            mlflow.log_params({k: v for k, v in model.get_params().items()
                               if isinstance(v, (int, float, str, bool)) or v is No
            mlflow.log_param("features", ",".join(features.columns))
        mlflow.log_metrics({f"backtest_{k}": v
                            for k, v in fold_metrics.drop(columns="fold").mean().it
        for _, row in fold_metrics.iterrows():
            mlflow.log_metric("fold_mae", row["mae"], step=int(row["fold"]))
```

The final model: one Pipeline

```
CATEGORICAL = ["season", "holiday", "workingday", "weather"]
features = ColumnTransformer(
    [
        ("categorical", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CATEGORICAL),
        ("hour", periodic_spline(24, 12), ["hour"]),
        ("weekday", periodic_spline(7, 3), ["weekday"]),
        ("month", periodic_spline(12, 6), ["month"])
    ],
    remainder="passthrough"
)
forecaster = Pipeline([
    ("features", features),
    ("model", HistGradientBoostingRegressor(loss="poisson", max_iter=40, learning_rate=0.05))
])
forecaster.fit(X_serve, y)  # backtest MAE about 40
mlflow.sklearn.log_model(forecaster, name="model", signature=signature,
                          input_example=X_serve.head(3), skops_trusted_types=TRUSTED)
```

- Raw columns in, forecast out: no lag bookkeeping at serving time

Tools worth knowing

- **statsmodels**: ARIMA/SARIMAX, ETS, STL, unobserved components (state space)
- **statsforecast** (Nixtla): fast AutoARIMA / AutoETS / Theta for many series
- **skforecast**, **sktime**, **darts**: scikit-learn-style wrappers for recursive/direct forecasting and backtesting
- **mlforecast**: lag features + LightGBM for thousands of series
- **Prophet**: additive trend + seasonality + holidays; easy to use, but often beaten by ETS or boosting in benchmarks

Summary and where to go from here

- Forecasting is regression with a clock: features must be available at forecast time
- Validate with time-ordered splits (`TimeSeriesSplit`, `gap`), and audit every feature
- Beat naive and seasonal naive first; report MAE/MASE, be careful with MAPE
- Gradient boosting on calendar, weather and lag features is a strong default; mind extrapolation
- Recursive vs. direct: decide by backtest; quantify uncertainty and check coverage
- Further: hierarchical forecasting, state space models, Gaussian processes, conformal intervals for time series, recurrent and convolutional networks (Week 6 MLPs extend to sequences)

Questions?

Backup

Time series classification

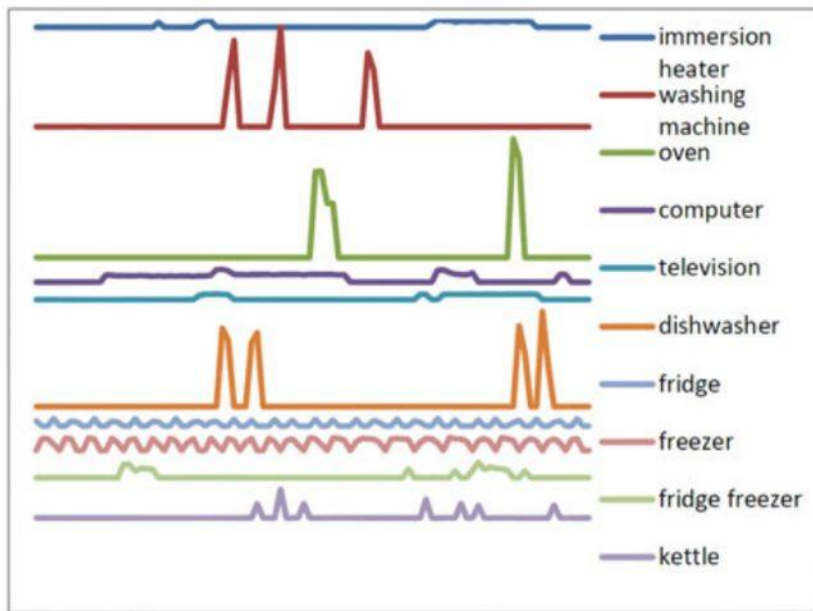


Fig. 1. Examples of daily profiles for the ten devices considered

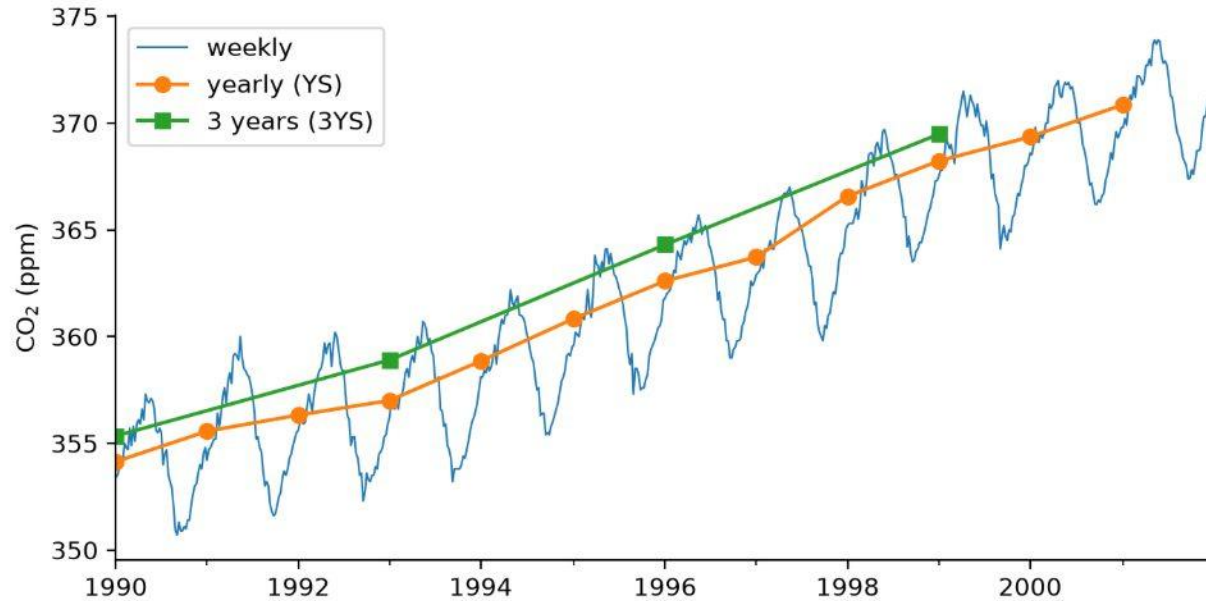
- Label a whole series (or a window): which appliance is running, which activity a phone sensor records

Filling gaps

```
weekly.ffill()    # carry the last observation forward  
weekly.bfill()    # carry the next observation backward: uses the future!  
weekly.interpolate(method="time")
```

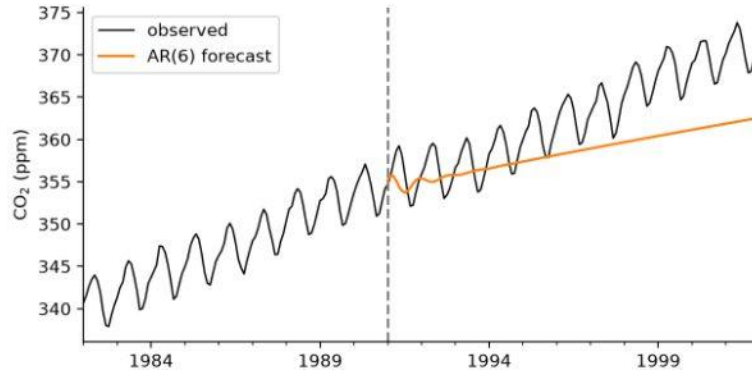
- Forward fill only uses the past; safe for forecasting features
- Back-fill and interpolation look ahead: fine for plots, leakage for features
- If you aggregate anyway (resample), you can often just drop missing values

Choosing the granularity

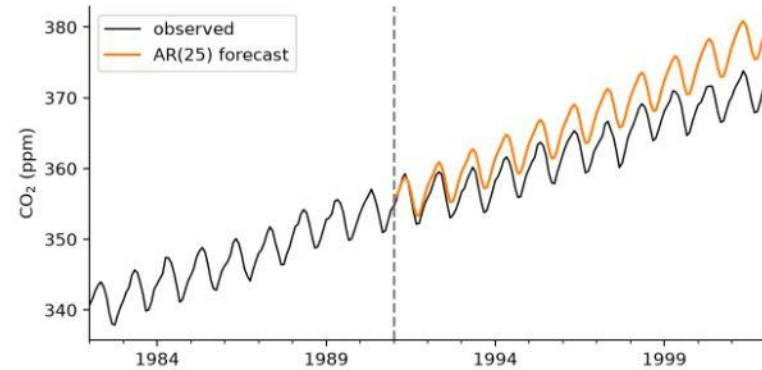


Impact of the order

`AutoReg(train, lags=6, trend="c")`



`AutoReg(train, lags=25, trend="c")`



Detrending with the trend model

```
residual = monthly - quad.predict(X[["t"]])
```

